

AnchorIQ 시스템 동작 흐름 — CS 심층 가이드

사용자가 "호르무즈 근처에 제재국 선박 있어?"를 입력하는 순간부터 답변이 화면에 나오기까지, 컴퓨터 내부에서 벌어지는 모든 것을 CS 관점에서 설명합니다.

Part 1: 네트워크 — 브라우저에서 서버까지

1.1 OSI 7계층 모델

컴퓨터 간 통신은 7개 계층으로 나뉩니다. 편지를 보내는 과정과 같습니다.

계층 7 — Application	(응용)	"편지 내용을 작성"
계층 6 — Presentation	(표현)	"한국어로 쓸지 영어로 쓸지"
계층 5 — Session	(세션)	"대화를 시작/유지/종료"
계층 4 — Transport	(전송)	"등기? 일반? 빠른우편?"
계층 3 — Network	(네트워크)	"어떤 경로로 보낼지"
계층 2 — Data Link	(데이터링크)	"바로 옆집에 전달"
계층 1 — Physical	(물리)	"전기 신호 / 광케이블"

AnchorIQ에서 실제로 어떻게 동작하는가

사용자가 `POST /api/ai/query` 를 보낼 때, 각 계층에서 벌어지는 일:

[계층 7 — Application Layer]

브라우저(Chrome)가 HTTP 요청을 생성합니다.

POST /api/ai/query HTTP/1.1

Host: localhost:8080

Content-Type: application/json

Cookie: access_token=eyJhbGciOiJIUzI1NiJ9...

{

"query": "호르무즈 근처에 제재국 선박 있어?"

}

이 계층에서 하는 일:

– HTTP 프로토콜 규약에 맞게 요청 메시지 구성

– URL 파싱 (프로토콜: http, 호스트: localhost, 포트: 8080, 경로: /api/ai/query)

– Cookie에서 JWT 토큰을 자동으로 포함 (Same-Origin이므로)

– Content-Type을 application/json으로 설정하여 서버에게 "본문이 JSON이야"라고 알림

[계층 6 — Presentation Layer]

데이터의 인코딩/압축/암호화를 처리합니다.

이 계층에서 하는 일:

– 한국어 "호르무즈"를 UTF-8로 인코딩

"호" → 0xED 0x98 0xB8 (3바이트)

"르" → 0xEB 0xA5 0xB4 (3바이트)

JSON 전체가 바이트 배열로 변환됨

– HTTPS라면 여기서 SSL/TLS 암호화가 수행됨

(현재 로컬은 HTTP이므로 암호화 없음)

– gzip 압축이 설정되어 있으면 여기서 압축

[계층 5 — Session Layer]

통신 세션을 관리합니다.

이 계층에서 하는 일:

– HTTP는 기본적으로 Stateless(상태 없음)이므로 세션 계층이 명시적으로 동작하지는 않음

– 하지만 TCP 연결의 Keep-Alive가 여기서 관리됨

– 브라우저가 이미 localhost:8080과 TCP 연결이 있으면 재사용 (Connection Pooling)

– 없으면 새 TCP 연결 수립

[계층 4 — Transport Layer (TCP)]

데이터의 신뢰성 있는 전송을 보장합니다.

이 계층에서 하는 일:

– TCP 세그먼트로 분할

HTTP 요청 데이터가 크면 여러 세그먼트로 나눔 (MSS: Maximum Segment Size, 보통 1460바이트)

– 포트 번호 부여

출발지 포트: 브라우저가 임의 선택 (예: 52341)

목적지 포트: 8080 (Spring Boot)

– 순서 번호(Sequence Number) 부여

세그먼트마다 번호를 매겨서 순서 보장

– 흐름 제어(Flow Control)

수신측이 처리할 수 있는 속도에 맞춰 전송

– 혼잡 제어(Congestion Control)

네트워크가 혼잡하면 전송 속도를 줄임

TCP 세그먼트 구조:

출발지 포트: 52341

목적지 포트: 8080

순서 번호: 1000

2바이트

2바이트

4바이트

확인 번호: 0	4바이트
플래그: PSH, ACK	"데이터 보냄, 이전 것 받았음"
윈도우 크기: 65535	수신 가능한 바이트 수
체크섬: 0xA3B2	데이터 무결성 검증
HTTP 요청 데이터 (페이로드)	
POST /api/ai/query HTTP/1.1\r\n...	

[계층 3 – Network Layer (IP)]

패킷을 목적지까지 라우팅합니다.

이 계층에서 하는 일:

- IP 헤더 추가
출발지 IP: 127.0.0.1 (loopback, 내 컴퓨터)
목적지 IP: 127.0.0.1 (같은 컴퓨터 내에서 통신)
- TTL(Time To Live) 설정: 64 (라우터 64개 통과 가능)
- localhost이므로 실제 라우팅 없이 loopback 인터페이스 사용
- 프로덕션이면 여기서 실제 라우터를 거쳐 AWS 서버까지 이동

IP 패킷 구조:

버전: IPv4	
헤더 길이: 20바이트	
전체 길이: 512바이트	
TTL: 64	
프로토콜: 6 (TCP)	
출발지 IP: 127.0.0.1	
목적지 IP: 127.0.0.1	
TCP 세그먼트 (위의 내용)	

[계층 2 – Data Link Layer]

같은 네트워크 내에서 프레임을 전달합니다.

이 계층에서 하는 일:

- localhost이므로 loopback 가상 인터페이스 사용
- 실제 네트워크 카드(NIC)를 거치지 않음
- 프로덕션이면 이더넷 프레임에 MAC 주소 추가
출발지 MAC: 00:1A:2B:3C:4D:5E
목적지 MAC: FF:FF:FF:FF:FF:FF (ARP로 조회)

[계층 1 – Physical Layer]

실제 전기/광 신호로 변환합니다.

- localhost이므로 물리 계층 없음 (메모리 내 데이터 복사)
- 프로덕션이면 이더넷 케이블의 전기 신호 또는 광케이블의 빛 신호

1.2 DNS (Domain Name System) 상세

DNS란 무엇인가

DNS는 분산 데이터베이스 시스템입니다. 전 세계 수십억 개의 도메인-IP 매핑을 계층적으로 관리합니다.

DNS 조회 과정 (재귀적 질의)

사용자가 api.anchoriq.com 을 입력했다고 가정합니다:

[Step 1] 브라우저 DNS 캐시 확인

Chrome 내부에 최근 조회 결과가 캐시되어 있는지 확인.
chrome://net-internals/#dns 에서 확인 가능.
캐시 TTL(Time To Live): 보통 60~300초.
→ 캐시 히트: 즉시 IP 반환 (0ms)
→ 캐시 미스: 다음 단계로

[Step 2] OS DNS 캐시 확인

운영체제(macOS)의 DNS 캐시 확인.
macOS: sudo dscacheutil -flushcache 로 초기화 가능.

/etc/hosts 파일도 확인:

```
127.0.0.1 localhost
→ localhost는 여기서 바로 해결됨 (127.0.0.1)
```

→ 캐시 히트: 즉시 반환
→ 캐시 미스: 다음 단계로

[Step 3] Local DNS Server (Resolver)에 질의

ISP(인터넷 서비스 제공자)의 DNS 서버에 질의.
또는 공용 DNS: Google 8.8.8.8, Cloudflare 1.1.1.1

질의: "api.anchoriq.com의 IP가 뭐야?"

이 DNS 서버도 캐시가 있음.
→ 캐시 히트: 반환
→ 캐시 미스: 다음 단계로 (재귀적 질의 시작)

[Step 4] Root DNS Server에 질의
전 세계 13개의 Root DNS 서버 (a.root-servers.net ~ m.root-servers.net)

질의: "api.anchoriq.com 아는 사람?"
응답: "나는 모르지만, .com을 담당하는 TLD 서버 주소를 알려줄게"
→ a.gtld-servers.net (192.5.6.30)

[Step 5] TLD (Top-Level Domain) DNS Server에 질의
.com, .net, .kr 등 최상위 도메인 서버

질의: "anchoriq.com 아는 사람?"
응답: "anchoriq.com의 네임서버는 ns1.example-registrar.com이야"
→ 203.0.113.1

[Step 6] Authoritative DNS Server에 질의
anchoriq.com 도메인을 실제로 관리하는 서버
(도메인 등록 시 설정한 네임서버)

질의: "api.anchoriq.com의 IP가 뭐야?"
응답: "52.78.123.45야" (AWS EC2 IP)
TTL: 300초 (5분간 캐시해도 됨)

[Step 7] 결과 반환 (역순)
Authoritative → Local DNS (캐시 저장) → OS (캐시 저장) → 브라우저 (캐시 저장)

총 소요 시간: 보통 20~100ms (캐시 미스 시)

DNS 레코드 종류

레코드 타입	용도	예시
A	도메인 → IPv4 주소	anchoriq.com → 52.78.123.45
AAAA	도메인 → IPv6 주소	anchoriq.com → 2001:db8::1
CNAME	도메인 → 다른 도메인	www.anchoriq.com → anchoriq.com
MX	메일 서버	anchoriq.com → mail.anchoriq.com
NS	네임서버	anchoriq.com → ns1.registrar.com
TXT	텍스트 정보	SPF, DKIM 등 메일 인증

AnchorIQ 로컬 환경에서의 DNS

localhost → 127.0.0.1

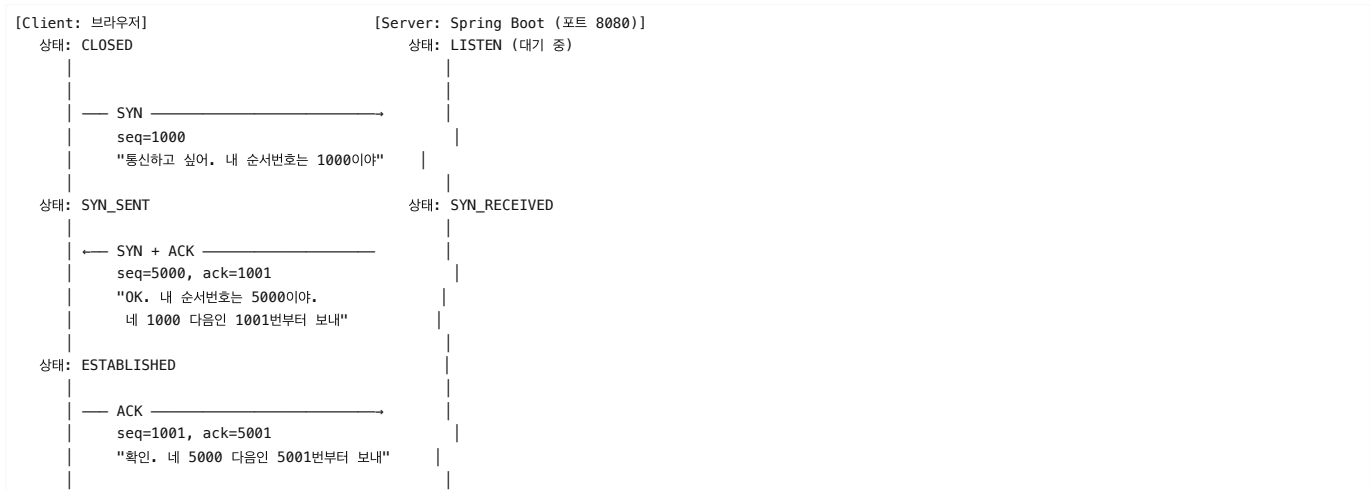
이것은 /etc/hosts 파일에 하드코딩되어 있어서
DNS 서버에 질의하지 않고 바로 해결됩니다.
→ DNS 조회 시간: 0ms (네트워크 통신 없음)

1.3 TCP 3-Way Handshake 상세

TCP가 신뢰성을 보장하는 방법

TCP(Transmission Control Protocol)는 UDP와 달리 데이터가 반드시 도착함을 보장합니다.

연결 수립 과정





왜 3번이나 악수하는가? (2번이면 안 되나?)

2-Way만 하면 생기는 문제:

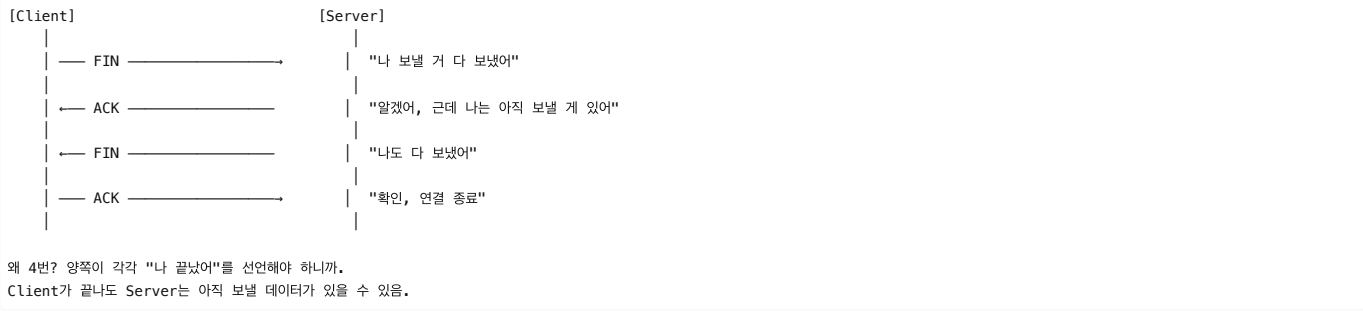
Client → SYN → Server

Client ← SYN+ACK ← Server

여기서 Client가 Server의 응답을 못 받았다면?
Server는 "연결됐다"고 생각하고 자원을 할당하지만,
Client는 연결이 안 됐다고 생각함.
→ Server 자원 낭비 (Half-Open Connection)

3-Way로 해결:
마지막 ACK로 Client가 "나도 확인했어"를 알려줌.
양쪽 모두 연결 상태를 확인한 후에만 통신 시작.

TCP 연결 종료 (4-Way Handshake)



1.4 포트(Port) 상세

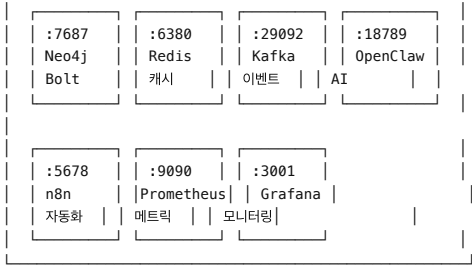
포트란 무엇인가

하나의 IP 주소에서 여러 프로세스가 동시에 통신할 수 있도록 하는 논리적 번호입니다.

포트 번호 범위: 0 ~ 65535 (16비트 정수)	
0 ~ 1023	: Well-Known Ports (시스템 예약) HTTP=80, HTTPS=443, SSH=22, FTP=21, DNS=53
1024 ~ 49151	: Registered Ports (애플리케이션 등록) PostgreSQL=5432, Neo4j=7687, Kafka=9092
49152 ~ 65535	: Dynamic/Ephemeral Ports (임시 포트) 브라우저가 요청할 때 OS가 자동 할당 예: 브라우저 → localhost:8080 요청 시 출발지 포트는 52341 같은 임시 포트

AnchorIQ의 포트 맵





소켓(Socket) = IP + 포트 조합
예: 127.0.0.1:8080 = Spring Boot 서버의 소켓
127.0.0.1:52341 = 브라우저의 임시 소켓

하나의 TCP 연결 = (출발지 IP:포트, 목적지 IP:포트) 4개의 조합으로 식별
예: (127.0.0.1:52341, 127.0.0.1:8080) = 이 연결은 고유함

1.5 Nginx 상세

Nginx란?

고성능 HTTP 서버이자 리버스 프록시입니다. 이벤트 기반(Event-Driven) 비동기 아키텍처로 10,000개 이상의 동시 연결을 처리할 수 있습니다.

Forward Proxy vs Reverse Proxy

[Forward Proxy] - 클라이언트 측 대리인

회사 내부 → (Proxy) → 인터넷

용도: 접속 차단, 캐싱, 익명화

예: 회사에서 유튜브 차단

[Reverse Proxy] - 서버 측 대리인 (Nginx)

인터넷 → (Nginx) → 내부 서버들

용도: 로드밸런싱, SSL, 보안, 캐싱

예: anchoriq.com → Nginx → Spring Boot

Nginx가 AnchorIQ에서 하는 일 (프로덕션)

```
[사용자]
|
| https://anchoriq.com/api/ai/query
▼
[Nginx (포트 443)]
|
| ① SSL/TLS 종료 (HTTPS 복호화)
|   클라이언트와 Nginx 사이만 암호화
|   Nginx → 내부 서버는 HTTP (빠름)
|
| ② 경로 기반 라우팅
|   /api/* → localhost:8080 (Spring Boot)
|   /*     → localhost:3004 (Next.js)
|   /ws/*  → localhost:8080 (WebSocket)
|
| ③ 로드 밸런싱 (서버가 여러 대일 때)
|   upstream backend {
|       server 10.0.1.1:8080 weight=3;  ← 3배 더 많이
|       server 10.0.1.2:8080 weight=1;
|       server 10.0.1.3:8080 backup;   ← 위 2개 죽으면 사용
|   }
|
| ④ Rate Limiting (DDoS 방어)
|   limit_req zone=api burst=20 nodelay;
|   → 초당 10개 이상 요청 시 429 반환
|
| ⑤ 보안 헤더 추가
|   X-Frame-Options: DENY
|   X-Content-Type-Options: nosniff
|   Strict-Transport-Security: max-age=31536000
|
| ⑥ 정적 파일 직접 서빙
|   /static/* → 파일 시스템에서 직접 전송
|   Spring Boot까지 안 가도 됨 (속도 향상)
|
| ⑦ 응답 압축 (gzip)
|   JSON 응답을 gzip으로 압축하여 전송
|   원본 10KB → 압축 2KB (80% 절약)
▼
[Spring Boot (포트 8080)]
```

Nginx vs Apache


```
"호르무즈 근처에 제재국 선박 있어?" 는 UTF-8로 62바이트.
(한글 1글자 = 3바이트, 영문/기호 = 1바이트)

[Cookie: access_token=eyJhbGci...]
JWT 토큰이 HttpOnly Cookie에 저장되어 있음.
브라우저가 Same-Origin (같은 도메인) 요청 시 자동으로 포함.
JavaScript로 접근 불가 (document.cookie에 안 보임) → XSS 방어.

[Accept: application/json]
"응답을 JSON으로 주세요." 콘텐츠 협상(Content Negotiation).
서버는 이 헤더를 보고 응답 형식을 결정.

[Origin: http://localhost:3004]
요청의 출처. CORS(Cross-Origin Resource Sharing) 검증에 사용.
서버가 이 Origin을 허용 목록에서 확인:
SecurityConfig.java → allowedOrigins: ["http://localhost:3004", ...]

[Connection: keep-alive]
TCP 연결을 응답 후에도 유지.
다음 요청 시 3-Way Handshake 없이 바로 재사용 → 속도 향상.
HTTP/1.1에서 기본값.
```

2.2 HTTP 응답 메시지 구조

AnchorIQ AI 질의 응답의 실제 HTTP 메시지

```
HTTP/1.1 200 OK\r\n                                     ← 상태 라인
Vary: Origin\r\n                                         ← CORS: Origin 별로 캐시 분리
Vary: Access-Control-Request-Method\r\n
Vary: Access-Control-Request-Headers\r\n
Access-Control-Allow-Origin: http://localhost:3004\r\n     ← CORS 허용
Access-Control-Allow-Credentials: true\r\n               ← 쿠키 포함 허용
X-Content-Type-Options: nosniff\r\n                       ← MIME 스니핑 방지
X-XSS-Protection: 1; mode=block\r\n                      ← XSS 필터 활성화
Cache-Control: no-cache, no-store\r\n                    ← 캐시 금지 (민감한 AI 응답)
X-Frame-Options: DENY\r\n                                ← iframe 삽입 방지 (Clickjacking)
Content-Type: application/json\r\n                       ← 응답 형식
Transfer-Encoding: chunked\r\n                          ← 분할 전송
Date: Sat, 04 Apr 2026 04:43:17 GMT\r\n
\r\n
{"success":true,"data":{"answer":"현재 조회 결과상 호르무즈 인근에...",...}}
```

보안 헤더 상세

```
[X-Content-Type-Options: nosniff]
  브라우저가 Content-Type을 무시하고 내용을 추측(스니핑)하는 것을 방지.
  예: text/plain으로 온 JavaScript를 실행하는 것을 차단.

[X-Frame-Options: DENY]
  이 페이지를 다른 사이트의 <iframe>에 넣을 수 없음.
  Clickjacking 공격 방지:
  - 공격자가 AnchorIQ 위에 투명 iframe을 올려놓고
  - 사용자가 "좋아요" 클릭 → 실제로는 "계정 삭제" 클릭
  → DENY로 원천 차단

[Strict-Transport-Security: max-age=31536000]
  HSTS (HTTP Strict Transport Security)
  "이 사이트는 앞으로 1년간 HTTPS로만 접속해라"
  → HTTP로 접속 시도해도 브라우저가 자동으로 HTTPS로 변환
  → SSL Strip 공격 방지

[Cache-Control: no-cache, no-store]
  AI 질의 응답은 민감할 수 있으므로 브라우저/프록시가 캐시하지 않도록.
  no-cache: 캐시하되, 매번 서버에 유효성 확인
  no-store: 아예 저장하지 마
  → 두 개를 같이 쓰면 "절대 캐시하지 마"
```

2.3 HTTP 메서드 심화

안전성(Safety)과 멍등성(Idempotency)

안전(Safe): 서버 상태를 변경하지 않는가?
멍등(Idempotent): 여러 번 실행해도 결과가 같은가?

메서드	안전?	멍등?	설명
GET	0	0	몇 번 조회해도 데이터 변경 없음
HEAD	0	0	GET과 같지만 본문 없이 헤더만 반환
POST	X	X	매번 새로운 리소스 생성 가능
PUT	X	0	같은 데이터로 10번 수정해도 결과 같음

DELETE	X	O	이미 삭제된 것을 또 삭제해도 결과 같음	
PATCH	X	X	부분 수정 (연산에 따라 결과 달라질 수 있음)	

AnchorIQ에서의 실제 사용

```
// GET - 안전 + 멍등: 일일 브리핑 조회 (서버 상태 안 바뀜)
@GetMapping("/api/ai/briefing")
public ResponseEntity<ApiResponse> getDailyBriefing() {
    return ResponseEntity.ok(ApiResponse.success(
        aiQueryApplicationService.getDailyBriefing()));
}

// POST - 비안전 + 비멍등: AI 질의 (API 사용량 증가, 로그 기록)
@PostMapping("/api/ai/query")
public ResponseEntity<ApiResponse> query(@RequestBody AiQueryRequest request) {
    // 매 호출마다 사용량 카운터 증가 → 비멍등
    Map<String, Object> result = service.handleQuery(request.query(), userId);
    return ResponseEntity.ok(ApiResponse.success(result));
}

// PUT - 비안전 + 멍등: 워크플로우 수정 (같은 데이터로 10번 해도 결과 같음)
@PutMapping("/api/workflows/{id}")
public ResponseEntity<ApiResponse> updateWorkflow(
    @PathVariable Long id, @RequestBody WorkflowUpdateRequest request) {
    return ResponseEntity.ok(ApiResponse.success(
        workflowService.update(id, request)));
}

// DELETE - 비안전 + 멍등: 북마크 삭제 (이미 없으면 404, 결과적으로 같음)
@DeleteMapping("/api/bookmarks/{id}")
public ResponseEntity<ApiResponse> deleteBookmark(@PathVariable Long id) {
    bookmarkService.delete(id);
    return ResponseEntity.ok(ApiResponse.success());
}
}
```

2.4 HTTP 상태 코드 심화

전체 분류

1xx - Informational (정보)
 100 Continue: "헤더는 받았으니 본문 보내"
 101 Switching Protocols: WebSocket 업그레이드 시

2xx - Success (성공)
 200 OK: 일반 성공
 201 Created: 리소스 생성 성공 (POST /api/auth/signup)
 204 No Content: 성공했지만 반환할 데이터 없음 (DELETE)

3xx - Redirection (리다이렉션)
 301 Moved Permanently: 영구 이동 (SEO 영향)
 302 Found: 임시 이동
 304 Not Modified: 캐시된 버전 사용해도 됨

4xx - Client Error (클라이언트 잘못)
 400 Bad Request: 잘못된 요청 형식
 예: {"query": ""} → query가 빈 문자열 → @NotBlank 위반

401 Unauthorized: 인증 안 됨 (로그인 필요)
 예: JWT 토큰 없이 /api/ai/query 호출
 Spring Boot SecurityConfig에서:
 .exceptionHandling(exception -> exception
 .authenticationEntryPoint((req, res, ex) -> {
 res.setStatus(401);
 res.getWriter().write("{\"code\":\"UNAUTHORIZED\"}");
 })))

403 Forbidden: 인증은 됐지만 권한 없음
 예: USER 역할이 /api/admin/** 접근 시도

404 Not Found: 리소스 없음
 예: GET /api/vessels/0000000 → 존재하지 않는 IMO

405 Method Not Allowed: 허용되지 않는 메서드
 예: GET /api/ai/query → POST만 허용

409 Conflict: 충돌
 예: 이미 존재하는 이메일로 회원가입

429 Too Many Requests: 요청 과다
 예: FREE 플랜 5회 AI 질의 초과
 → PlanLimitExceededException → 429

5xx - Server Error (서버 잘못)

500 Internal Server Error: 예상치 못한 서버 오류
예: Neo4j 연결 끊김, NullPointerException

502 Bad Gateway: 프록시(Nginx)가 백엔드에서 잘못된 응답 받음
예: Spring Boot가 죽어있을 때 Nginx가 반환

503 Service Unavailable: 서버 점검 또는 과부하

504 Gateway Timeout: 프록시가 백엔드 응답을 시간 내에 못 받음
예: AI 질의가 60초 넘게 걸릴 때

2.5 CORS (Cross-Origin Resource Sharing) 상세

왜 필요한가

동일 출처 정책 (Same-Origin Policy):
브라우저는 보안상 다른 출처(Origin)의 리소스 접근을 차단합니다.

Origin = 프로토콜 + 호스트 + 포트

http://localhost:3004 (프론트엔드)
http://localhost:8080 (백엔드)
→ 포트가 다르므로 다른 Origin!
→ 프론트에서 백엔드 API 호출이 기본적으로 차단됨!
→ CORS 설정으로 허용해야 함

CORS 동작 과정

[Preflight 요청 - 사전 점검]

Simple Request가 아닌 경우 (POST + application/json은 Simple이 아님),
브라우저가 먼저 OPTIONS 요청을 보냄:

OPTIONS /api/ai/query HTTP/1.1
Origin: http://localhost:3004
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type

Spring Boot SecurityConfig에서:
.requestMatchers(HttpMethod.OPTIONS, "/*").permitAll()
→ OPTIONS 요청은 인증 없이 통과

서버 응답:
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:3004
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: *
Access-Control-Allow-Credentials: true ← Cookie 포함 허용
Access-Control-Max-Age: 3600 ← 1시간 캐시

브라우저: "OK, 허용했으니 실제 요청 보내도 됨"

[실제 요청]
POST /api/ai/query HTTP/1.1
Origin: http://localhost:3004
Cookie: access_token=eyJ...
{"query": "호르무즈..."}

AnchorIQ CORS 설정 (SecurityConfig.java)

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();

    // 허용할 Origin (프론트엔드 주소들)
    config.setAllowedOrigins(List.of(
        "http://localhost:3000", // Next.js 기본
        "http://localhost:3002", // 대체 포트
        "http://localhost:3004", // 현재 사용 중
        "http://localhost:5173" // Vite 개발 서버
    ));

    // 허용할 HTTP 메서드
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));

    // 허용할 헤더 (* = 모두)
    config.setAllowedHeaders(List.of("*"));

    // Cookie 전송 허용 (JWT가 Cookie에 있으므로 필수!)
    config.setAllowCredentials(true);
}
```

```
// Preflight 캐시 시간 (1시간 동안 OPTIONS 재요청 안 함)
config.setMaxAge(3600L);

UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
source.registerCorsConfiguration("/**", config);
return source;
}
```

Part 3: Spring Boot 내부 동작

3.1 Spring IoC 컨테이너

IoC (Inversion of Control) — 제어의 역전

[IoC 없이 – 개발자가 직접 생성]

```
public class AiQueryController {
    // 개발자가 직접 new로 생성
    private AiQueryApplicationService service = new AiQueryApplicationServiceImpl(
        new NaturalLanguageQueryServiceImpl(
            new OpenClawClient(new WebClient(...)),
            new Neo4jClient(...)
        ),
        new SubscriptionServiceImpl(new JpaSubscriptionRepository(...)),
        new StringRedisTemplate(new RedisConnectionFactory(...))
    );
    // → 의존성이 바뀌면 여기도 다 바꿔야 함
    // → 테스트할 때 Mock으로 교체 불가능
}
```

[IoC 있어 – Spring 컨테이너가 생성 및 주입]

```
@RestController
@RequiredArgsConstructor // Lombok: final 필드를 생성자로 주입
public class AiQueryController {
    private final AiQueryApplicationService service;
    // → Spring이 알아서 구현체를 찾아 주입
    // → 테스트 시 Mock으로 쉽게 교체 가능
}
```

Bean 생명주기

Spring Boot 시작 시:

- 컴포넌트 스캔
@Component, @Service, @Repository, @Controller, @Configuration
어노테이션이 붙은 클래스를 모두 찾음
- Bean 정의 등록
각 클래스를 Bean Definition으로 등록
- 의존성 해결 (DI)
각 Bean이 필요로 하는 의존성을 분석
AiQueryController → AiQueryApplicationService 필요
AiQueryApplicationServiceImpl → NaturalLanguageQueryService 필요
NaturalLanguageQueryServiceImpl → AiClient, Neo4jClient 필요
- Bean 인스턴스 생성 (의존성 순서대로)
① Neo4jClient 생성
② OpenClawClient 생성
③ NaturalLanguageQueryServiceImpl 생성 (①② 주입)
④ AiQueryApplicationServiceImpl 생성 (③ 주입)
⑤ AiQueryController 생성 (④ 주입)
- 초기화 콜백
@PostConstruct 메서드 실행
- 준비 완료
"Started AnchoriqApplication in 15.2s"

3.2 Spring MVC 요청 처리 파이프라인

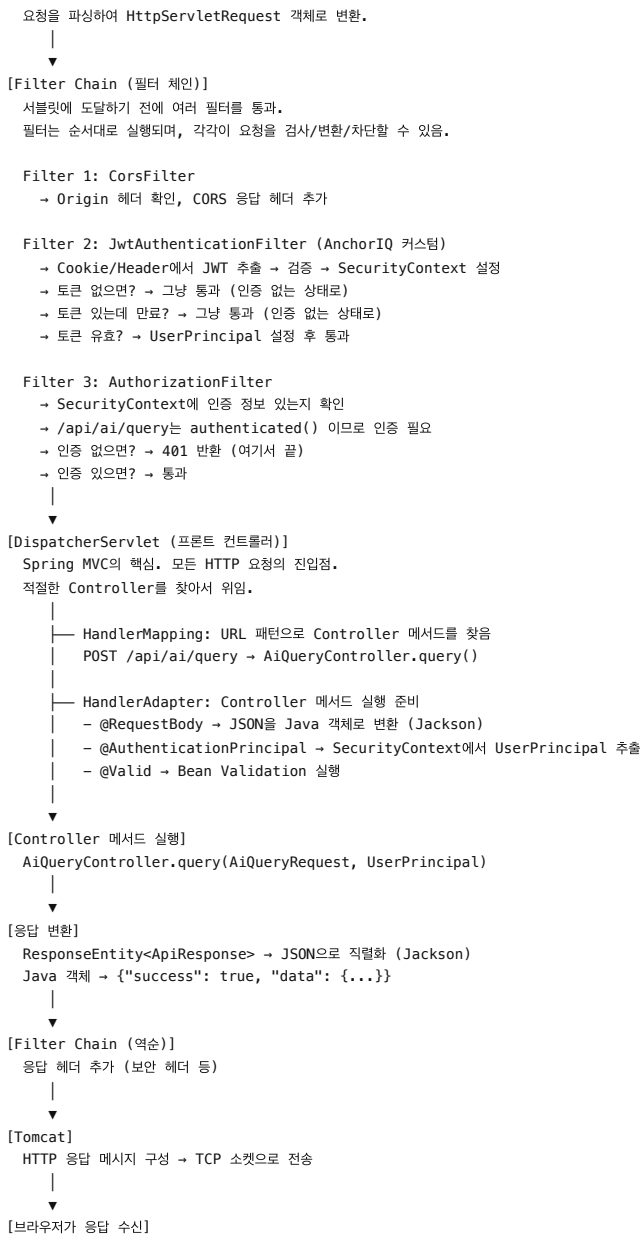
HTTP 요청이 Controller에 도착하기까지

[HTTP 요청 수신]



[Tomcat (내장 웹 서버)]

Spring Boot에 내장된 Tomcat이 TCP 소켓에서 HTTP 요청을 읽음.
포트 8080에서 LISTEN 중.



@RequestBody 역직렬화 과정

HTTP 본문 (JSON 문자열):

```
{"query": "호르무즈 근처에 제재국 선박 있어?"}
```

① `Jackson ObjectMapper`가 JSON 파싱
② `Java Record`의 필드 이름과 JSON 키를 매칭
"query" → `AiQueryRequest.query`
③ 값 할당
query = "호르무즈 근처에 제재국 선박 있어?"

결과: `AiQueryRequest(query="호르무즈 근처에 제재국 선박 있어?")`

```
// AiQueryRequest.java
public record AiQueryRequest(
    @NotBlank String query    // @NotBlank: null이거나 빈 문자열이면 400 예러
) {}

// @Valid가 Controller에 있으면 자동 검증:
// - query가 null → MethodArgumentNotValidException → 400 Bad Request
// - query가 ""   → MethodArgumentNotValidException → 400 Bad Request
// - query가 "호르무즈..." → 통과
```

3.3 AOP (Aspect-Oriented Programming)

@Transactional의 동작 원리

```
@Service
public class SubscriptionServiceImpl {
    @Transactional
    public void incrementApiUsage(Long userId) {
        Subscription sub = repository.findById(userId);
        sub.incrementUsage();
        repository.save(sub);
    }
}
```

```
public class SubscriptionServiceImpl$$$SpringCGLIB$$Proxy {
    public void incrementApiUsage(Long userId) {
        // ㉑ 트랜잭션 시작
        TransactionStatus tx = transactionManager.getTransaction();
        try {
            // ㉒ 원본 메서드 실행
            Subscription sub = repository.findById(userId);
            sub.incrementUsage();
            repository.save(sub);

            // ㉓ 정상 완료 → 커밋
            transactionManager.commit(tx);
        } catch (Exception e) {
            // ㉔ 예외 발생 → 롤백
            transactionManager.rollback(tx);
            throw e;
        }
    }
}
```

핵심 로직 (incrementUsage)과 횡단 관심사 (트랜잭션)를 분리.
개발자는 비즈니스 로직만 작성하고,
트랜잭션 시작/커밋/롤백은 Spring AOP가 자동 처리.

```
@Service
public class SomeService {

    @Transactional
    public void methodA() {
        // 트랜잭션 적용됨 ✅
    }

    public void methodB() {
        this.methodA(); // 프록시를 거치지 않으므로 트랜잭션 안 걸림 ❌
        // this는 프록시가 아닌 실제 객체를 가리킴
    }
}
```

해결: methodA()를 별도 클래스로 분리.

→ AnchorIQ에서 Application Service와 Domain Service를 분리한 이유 중 하나.

4.1 JWT 내부 구조

```
실제 JWT: eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJ1bmVtYWlsIjoieWl0ZXN0QGUiOjY2ZyIsImh0dCI6MTcxMjE5NmZmZWxzX2hwaWJvbnR5cyIsImF1dG8iOiJ1bmVtYWlsInQ9.LmVybSIsInJvbGU0IjVJU0VSIGF1dG8iOiJ1bmVtYWlsInQ9.LmVybSIsInJvbGU0IjVJU0VSIGF1dG8iOiJ1bmVtYWlsInQ9
```

점(.)으로 3개 부분 분리 후 Base64URL 디코딩:

```
[1. Header]
Base64URL: eyJhbGciOiJIUzI1NiJ9
디코딩:    {"alg":"HS256"}

alg: 서명 알고리즘
- HS256 = HMAC-SHA256 (대칭키, AnchorIQ에서 사용)
- RS256 = RSA-SHA256 (비대칭키, 마이크로서비스에서 주로 사용)
```

```
[2. Payload (Claims)]
Base64URL: eyJlc2VySWQiOiJue...
디코딩:    {
      "userId": 5,
      "email": "aitest@anchoriq.com",
      "role": "USER",
    }
```

```
    "type": "access",
    "iat": 1712197375,      ← 발급 시각 (Unix Timestamp)
    "exp": 1712215375      ← 만료 시각 (18000초 = 5시간 후)
  }
```

주요 클레임:

- `userId`: 사용자 식별자 (DB PK)
- `email`: 이메일
- `role`: 권한 (USER, ADMIN)
- `type`: `access`(API 호출용) vs `refresh`(갱신용) 구분
- `iat` (issued at): 발급 시각
- `exp` (expiration): 만료 시각

[3. Signature]

```
HMAC-SHA256(
  base64url(header) + "." + base64url(payload),
  "anchoriq-jwt-secret-key-2026-must-be-at-least-256-bits-long!!"
)
```

서명 검증 과정:

- ① 서버가 `header` + `payload`로 서명을 재계산
- ② 전달받은 `signature`와 비교
- ③ 일치하면 → 위조되지 않음 
- ④ 불일치하면 → 위조됨  → 401 반환

왜 Base64URL이지 암호화가 아닌가?

중요: JWT의 `payload`는 암호화가 아닙니다!

Base64는 인코딩(형식 변환)일 뿐, 누구나 디코딩할 수 있습니다.

→ JWT에 비밀번호, 카드번호 등을 넣으면 안 됩니다.

→ `userId`, `email`, `role` 정도만 넣는 이유.

JWT가 보장하는 것:

-  무결성 (Integrity): 내용이 변조되지 않았음 (서명으로 검증)
-  인증 (Authentication): 이 토큰을 발급한 건 우리 서버
-  기밀성 (Confidentiality): 내용을 숨기지 않음

기밀성이 필요하면: JWE (JSON Web Encryption) 사용

AnchorIQ에서는 JWE 불필요 (`payload`에 민감 정보 없음)

BCrypt 비밀번호 해싱

회원가입 시:

원문: "Test1234!"

해시: "\$2a\$10\$Wkd.J2kfGWgyj18VnSieB0WZTbtHRJBSJIoUpqX0oAHjwu9jXzAhG"

\$2a\$: BCrypt 알고리즘

10\$: Cost Factor ($2^{10} = 1024$ 회 반복)

Wkd.J2kfGWgyj18VnSieB0: Salt (22문자, 랜덤 생성)

WZTbtHRJBSJ...: 해시 결과

로그인 시:

- ① 사용자가 "Test1234!" 입력
- ② DB에서 해시값 조회
- ③ BCrypt가 같은 Salt로 입력값을 해시
- ④ 해시 결과 비교 → 일치하면 로그인 성공

왜 SHA-256이 아니라 BCrypt?

- SHA-256은 빠름 → 무차별 대입 공격에 취약
- BCrypt는 의도적으로 느림 (Cost Factor로 조절)
- Cost 10이면 ~100ms, Cost 12면 ~400ms
- 해커가 1초에 100억 개 시도 → BCrypt면 10개만 시도 가능

Part 5: 데이터베이스 심화

5.1 PostgreSQL — ACID 트랜잭션

ACID란?

A — Atomicity (원자성)

"전부 성공하거나 전부 실패하거나"

예: 결제 처리

- ① 잔액 차감: 10,000원 → 0원
- ② 구독 활성화: FREE → PRO

②에서 에러 나면? → ①도 롤백 (원래 10,000원으로 복원)

반만 적용되는 일은 없음.

C — Consistency (일관성)

"규칙을 항상 만족"	
예: 잔액은 음수가 될 수 없음 (CHECK 제약조건)	
→ 잔액 0원에서 결제 시도 → 제약조건 위반 → 트랜잭션 거부	
I – Isolation (격리성)	
"동시 실행되는 트랜잭션이 서로 간섭하지 않음"	
예: 두 사용자가 동시에 같은 선박 워크플로우 수정	
트랜잭션 A: 워크플로우 읽기 → 수정 → 저장	
트랜잭션 B: 워크플로우 읽기 → 수정 → 저장	
격리 수준에 따라:	
– READ_COMMITTED (PostgreSQL 기본): B가 A의 커밋 전 데이터를 못 봄	
– SERIALIZABLE: A가 끝날 때까지 B가 대기	
D – Durability (지속성)	
"커밋된 데이터는 영구 보존"	
서버가 갑자기 꺼져도 커밋된 데이터는 디스크에 저장되어 있음.	
→ WAL (Write-Ahead Log): 변경 사항을 먼저 로그에 기록 후 실제 반영	
→ 서버 재시작 시 WAL을 읽어 미반영 데이터 복구	

인덱스 (B-Tree)

인덱스 없이 검색 (Full Table Scan):
SELECT * FROM users WHERE email = 'aitest@anchoriq.com';
→ 100만 행을 처음부터 끝까지 순차 탐색
→ $O(n)$ = 100만 번 비교
→ 느림
인덱스 있어 (B-Tree Index):
CREATE INDEX idx_users_email ON users(email);
→ B-Tree 구조로 정렬된 인덱스 생성
→ $O(\log n) = \log_2(1,000,000) \approx 20$ 번 비교
→ 100만 배 빠름!
B-Tree 구조:
[M] [E] [S] [A-D] [F-L] [N-R] [T-Z] [aitest@...] ← 3~4단계만에 도착!
인덱스의 단점:
– INSERT/UPDATE 시 인덱스도 갱신 → 쓰기 성능 저하
– 디스크 공간 추가 사용
– 카디널리티(고유값 수)가 낮은 컬럼은 효과 없음
예: gender(M/F) 인덱스 → 50%씩이라 Full Scan과 차이 없음

5.2 Neo4j — 그래프 데이터베이스

Index-Free Adjacency

RDBMS에서 관계 탐색:
SELECT v.name FROM vessels v
JOIN companies c ON v.company_id = c.id
JOIN countries co ON c.country_id = co.id
WHERE co.sanctioned = true;
동작:
① vessels 테이블 스캔 → company_id 찾기
② companies 인덱스에서 company_id 검색 → $O(\log n)$
③ countries 인덱스에서 country_id 검색 → $O(\log n)$
→ JOIN이 늘어날수록 느려짐: $O(n \times \log n \times \log n)$
Neo4j에서 관계 탐색:
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:REGISTERED_IN]->(co:Country)
WHERE co.sanctioned = true
RETURN v.name;
동작:
① 각 Vessel 노드가 인접 노드의 물리적 포인터를 직접 가짐
② 포인터를 따라가기만 하면 됨 → $O(1)$ per hop
③ 4-hop 탐색 = $O(4)$ = 상수 시간!
→ 데이터가 아무리 많아도 관계 탐색 속도는 동일
이것이 Index-Free Adjacency:
인덱스 조회 없이 물리적 포인터로 직접 이동.
그래프 DB가 관계 탐색에서 RDBMS보다 압도적으로 빠른 이유.

Cypher 쿼리 실행 과정

```
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:REGISTERED_IN]->(co:Country)
    -[:SANCTIONED_BY]->(s:Sanction)
WHERE toLower(cp.name) CONTAINS 'hormuz'
RETURN v.name, co.name, s.referenceNumber
```

실행 계획 (EXPLAIN):

- ① NodeByLabelScan: Chokepoint 라벨의 모든 노드 스캔
→ name에 'hormuz' 포함 확인 (Filter)
→ 결과: 1개 (Strait of Hormuz)
- ② Expand(All): Hormuz에서 PASSES_THROUGH 관계를 역추적
→ Route 노드들 찾기
- ③ Expand(All): Route에서 SAILING_ON 관계를 역추적
→ Vessel 노드들 찾기
- ④ Expand(All): Vessel에서 OWNED_BY 관계 따라가기
→ Company 노드 찾기
- ⑤ Expand(All): Company에서 REGISTERED_IN 관계 따라가기
→ Country 노드 찾기
- ⑥ Expand(All): Country에서 SANCTIONED_BY 관계 따라가기
→ Sanction 노드 찾기
- ⑦ Filter: 모든 조건을 만족하는 경로만 반환
- ⑧ Projection: v.name, co.name, s.referenceNumber만 추출하여 반환

Part 6: Kafka 심화

6.1 Kafka 내부 동작

파티션(Partition)과 오프셋(Offset)

토픽: ais-positions (3개 파티션)

Partition 0: [msg0] [msg3] [msg6] [msg9] ...
 off=0 off=1 off=2 off=3

Partition 1: [msg1] [msg4] [msg7] [msg10] ...
 off=0 off=1 off=2 off=3

Partition 2: [msg2] [msg5] [msg8] [msg11] ...
 off=0 off=1 off=2 off=3

파티션(Partition):

- 토픽을 물리적으로 나눈 단위
- 각 파티션은 독립적인 로그 파일
- 파티션 수 = 병렬 처리 수 (3개 파티션 → 3개 Consumer 동시 처리)

오프셋(Offset):

- 파티션 내에서 메시지의 순서 번호
- Consumer가 "어디까지 읽었는지" 추적하는 데 사용
- Consumer가 죽었다가 살아나도 마지막 오프셋부터 재개

키(Key) 기반 파티셔닝:

AIS 메시지의 키 = MMSI (선박 식별 번호)

같은 MMSI는 항상 같은 파티션으로 → 순서 보장

예: MMSI 265059850의 위치 업데이트는 항상 Partition 0으로

Consumer Group

Consumer Group: "neo4j-updater"

Consumer A → Partition 0 담당

Consumer B → Partition 1 담당

Consumer C → Partition 2 담당

Consumer Group: "redis-writer"

Consumer D → Partition 0 담당

Consumer E → Partition 1, 2 담당

같은 그룹 내에서는 하나의 파티션을 하나의 Consumer만 읽음 (부하 분산)

다른 그룹은 독립적으로 같은 메시지를 읽음 (다중 소비)

→ AIS 위치 데이터를 Neo4j에도 저장하고, Redis에도 저장하고, AI에도 전달

각각 독립적인 Consumer Group으로 동시 처리

Part 7: Docker 심화

7.1 컨테이너 vs 가상머신

[가상머신 (VM)]

VM 1

Application

Libraries

Guest OS (Ubuntu 22)

← 전체 OS 포함 (수 GB)

Hypervisor

Host OS (macOS)

Hardware

시작 시간: 분 단위
메모리: GB 단위

[Docker 컨테이너]

Container 1

Application

Libraries

← OS 커널 공유 (수 MB)

Docker Engine

Host OS (macOS)

Hardware

시작 시간: 초 단위
메모리: MB 단위

핵심 차이:
VM: 하드웨어를 가상화 → 전체 OS 필요 → 무거움
Docker: OS 커널을 공유 → 프로세스 격리만 → 가벼움

Docker 이미지 레이어

neo4j:5-community 이미지 구조:

Layer 1: [Base OS]

debian:bullseye-slim (50MB)

Layer 2: [Java JRE]

OpenJDK 17 (200MB)

Layer 3: [Neo4j]

Neo4j Community 5.x (300MB)

Layer 4: [Config]

neo4j.conf (1KB)

각 레이어는 읽기 전용(Read-Only).
컨테이너 실행 시 최상위에 쓰기 가능(Read-Write) 레이어 추가.
→ 여러 컨테이너가 같은 이미지를 공유 (디스크 절약)

부록: 전체 흐름 한눈에 보기

[사용자] "호르무즈 근처에 제재국 선박 있어?" 입력

① DNS: localhost → 127.0.0.1

② TCP 3-Way Handshake (이미 연결되어 있으면 생략)

③ HTTP POST 요청 전송

Content-Type: application/json

Cookie: access_token=eyJ... (HttpOnly)

▼

[Tomcat (포트 8080)] HTTP 요청 수신

④ JwtAuthenticationFilter

Cookie에서 JWT 추출 → HMAC-SHA256 서명 검증

→ userId=5, role=USER 추출 → SecurityContext 설정

⑤ AuthorizationFilter

/api/ai/query는 authenticated() → 인증 확인 ✅

⑥ DispatcherServlet → HandlerMapping

POST /api/ai/query → AiQueryController.query()

⑦ Jackson: JSON → AiQueryRequest 역직렬화

▼

[AiQueryController] – Layer 1: Controller

⑧ service.handleQuery("호르무즈...", userId=5)

▼

[AiQueryApplicationServiceImpl] – Layer 2: Application Service


```
| ⑨ 쿼터 확인 (PostgreSQL: subscription 테이블)  
| ⑩ Redis 캐시 확인 (GET ai:result:hashcode → MISS)  
| ⑪ queryService.executeQuery("호르무즈...")  
▼  
[NaturalLanguageQueryServiceImpl] - Layer 3: Domain Service  
|  
| ⑩ 1차 AI 호출: 자연어 → Cypher  
▼  
[OpenClawClient] - HTTP POST to 127.0.0.1:18789  
|  
| ⑩ OpenClaw → OpenAI GPT 엔진  
|   System Prompt: Neo4j 스키마 + 규칙  
|   User: "호르무즈 근처에 제재국 선박 있어?"  
|   Temperature: 0.1 (정확한 Cypher 생성)  
|  
| ⑩ AI 응답: Cypher 쿼리  
▼  
[CypherQueryValidator] - 보안 검증  
|  
| ⑩ CREATE/DELETE/SET 등 쓰기 명령 차단  
|   MATCH/RETURN만 허용 ✅  
▼  
[Neo4jClient] - Bolt 프로토콜 (포트 7687)  
|  
| ⑩ Cypher 실행  
|   Vessel → OWNED_BY → Company → REGISTERED_IN →  
|   Country → SANCTIONED_BY → Sanction  
|   + PASSES_THROUGH → Chokepoint (Hormuz)  
|  
| ⑩ 결과: 0건 (조건 충족 선박 없음)  
▼  
[NaturalLanguageQueryServiceImpl]  
|  
| ⑩ 2차 AI 호출: 쿼리 결과 → 자연어  
▼  
[OpenClawClient] - HTTP POST to 127.0.0.1:18789  
|  
| ⑩ AI 응답: "현재 조회 결과상 호르무즈 인근에  
|   제재 대상국 관련 선박은 확인되지 않았습니다..."  
▼  
[AiQueryApplicationServiceImpl]  
|  
| ⑩ Redis 캐싱: SET ai:result:hash → 5분 TTL  
| ⑩ PostgreSQL: API 사용량 +1 (@Transactional)  
| ⑩ Elasticsearch: 로그 기록 (비동기, 실패 무시)  
▼  
[AiQueryController]  
|  
| ⑩ Jackson: Map → JSON 직렬화  
| ⑩ ResponseEntity.ok(ApiResponse.success(result))  
▼  
[Tomcat] HTTP 200 OK 응답 전송  
|  
| ⑩ TCP로 응답 데이터 전송  
|   브라우저가 ACK 보내면 완료  
▼  
[브라우저]  
|  
| ⑩ JavaScript가 JSON 파싱  
| ⑩ React 컴포넌트가 화면에 렌더링  
▼  
[사용자가 답변 확인]  
"현재 조회 결과상 호르무즈 인근에 제재 대상국 관련 선박은  
확인되지 않았습니다..."
```

총 27단계. DNS 조회부터 화면 렌더링까지. 소요 시간: 약 2~5초 (대부분 AI 호출 2번에서 소요)

이 문서는 AnchorIQ의 실제 코드, 설정, 테스트 결과를 기반으로 작성되었습니다.